Package and exception handling =>

import java.lang.*;

- it is a fundamental class and all the methods i.e System and many more are in this package only
- We do not explicitly write this package as it is always imported implicitly

But Scanner class is not in this package. There is one more package which name is import java.util.Scanner;

What are packages => It is like a folder that hold together group of related classes.

Need for Packages =>
1. Organization
2. Reusability
3. Namespace Management
4. Security
5. Modularity

Types of Packages =>
1. Built-in Packages
2. User-defined packages

↓

Pre-defined packages provided by the Java Development Kit (JDK)

Essential Build in Packages

1. java.lang → functionalities like String, Math, System

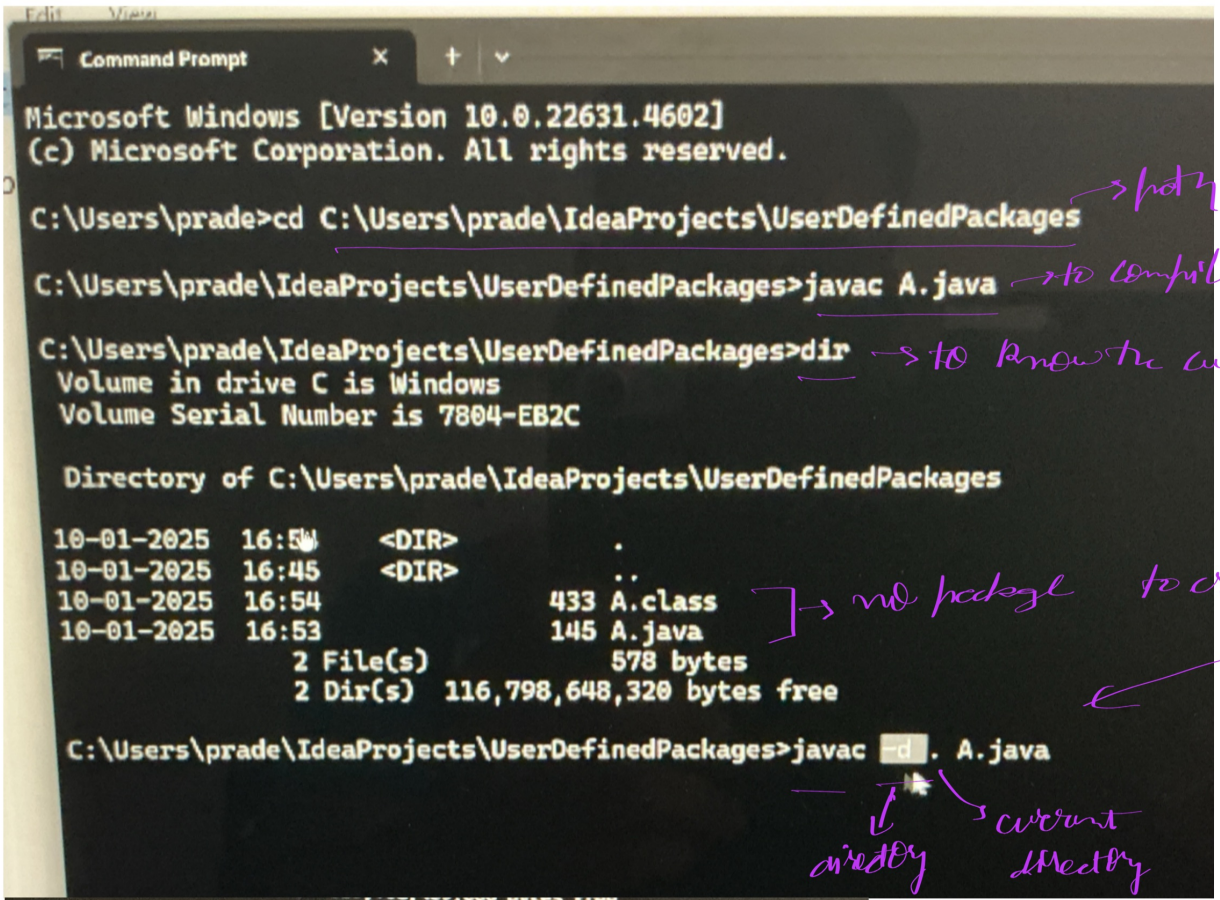
② java.io → input/output operations, handling files and streams

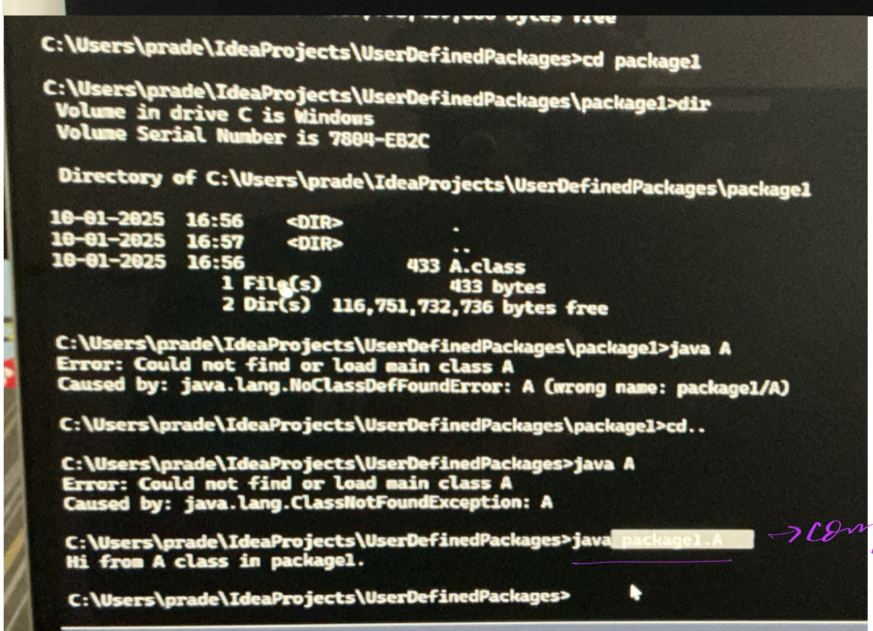
③ java.util → utility classes like Date, Calendar, Collections

④ java.sql → connect to and interacting with database

⑤ java.net → network programming, handling sockets and URLs

⑥ java.awt → Graphical user interface (GUI) interface





Java User-Defined Packages

Complete practical guide: concepts, syntax, examples, compilation, imports, access modifiers, subpackages, JARs, troubleshooting and exercises

Learning goal

Learn how to create your own Java package, organize source files, compile and run classes, import package types, understand access rules, and troubleshoot common errors.

Requirements: a JDK and a terminal or Java IDE. Examples work with modern Java versions.

Contents

• 1. What a package is and why it is useful

• 2. Package declaration rules

• 3. Build a package step by step

• 4. Importing and fully qualified names

• 5. A multi-package example

• 6. Access modifiers

• 7. Subpackages

• 8. Compilation, classpath and execution

• 9. IDE, Maven and Gradle layouts

• 10. Creating and using a JAR

• 11. Common errors

• 12. Best practices

• 13. Practice questions and answers

• 14. Quick revision sheet

1. What is a package?

A **package** is a namespace that groups related Java classes, interfaces, enums, annotations and other types. Built-in examples include `java.util`, `java.io` and `java.time`. A **user-defined package** is a package you create for your own code.

```
package college.student;

public class Student {
    public void show() {
        System.out.println("Student class");
    }
}
```

The package name here is `college.student`. Conventionally, package names use lowercase letters. Large projects often use a reversed domain name such as `com.example.billing`.

Package name and directory path

Dots in a package name correspond to directories under the source root. A class in package `college.student` is normally stored in `college/student/Student.java` relative to the source root.

2. Why create user-defined packages?

• **Organization:** group related classes, such as `model`, `service` and `util`.

• **Avoid naming conflicts:** `school.Student` and `college.Student` can be distinct classes.

• **Access control:** package-private members are accessible to code in the same package, subject to Java's normal rules.

• **Reuse:** package classes can be reused by other application code or distributed in a JAR.

• **Maintainability:** package structure makes larger applications easier to navigate.

3. Package declaration rules

• If present, the package declaration must be the first non-comment statement in the source file.

• A source file can contain at most one package declaration.

• Imports appear after the package declaration and before type declarations.

• The package directory structure should match the declaration for reliable compilation and IDE support.

• Without a package declaration, a class belongs to the unnamed/default package. Avoid this for reusable or substantial applications.

```
package com.example.app;  // package declaration
import java.util.List;    // imports follow
public class Main {       // then the type
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

4. Build your first package from scratch

We will create `mypack.Message` and use it from a separate package named `app`.

Step 1 — Create the folders

```
PackageDemo/
■■■ src/
■   ■■■ mypack/Message.java
■   ■■■ app/Main.java
■■■ out/  (compiled classes)
```

```
mkdir -p src/mypack src/app out
```

Step 2 — Create Message.java

```
/* File: src/mypack/Message.java */
package mypack;

public class Message {
    public void display() {
        System.out.println("Hello from mypack!");
    }
}
```

The declaration `package mypack;` puts the class in that package. The class is `public` because code in another package must be able to access it.

Step 3 — Create Main.java

```
/* File: src/app/Main.java */
package app;

import mypack.Message;

public class Main {
    public static void main(String[] args) {
        Message message = new Message();
        message.display();
    }
}
```

The import lets us use the short name `Message`. Because the classes are in different packages, both the class and the called method must be accessible from outside `mypack`.

Step 4 — Compile

```
javac -d out src/mypack/Message.java src/app/Main.java
```

`javac` compiles the source. The `-d out` option writes class files under `out`, preserving package directories. You should get `out/mypack/Message.class` and `out/app/Main.class`.

Step 5 — Run

```
java -cp out app.Main
```

Output:

```
Hello from mypack!
```

Run the fully qualified class name `app.Main`; the classpath root is `out`, the directory above the package folders.

5. Importing classes and fully qualified names

Import one class

```
import mypack.Message;
Message m = new Message();
m.display();
```

Wildcard import

```
import mypack.*;
Message m = new Message();
```

The wildcard makes accessible types directly inside `mypack` available by simple name. It does **not** import types from subpackages.

Fully qualified class name

```
mypack.Message m = new mypack.Message();
m.display();
```

• Imports make names easier to write; they do not copy code into the file.

• Classes in the same package normally do not need to import one another.

• Types in `java.lang`, including `String` and `Math`, are available without explicit imports.

• If two imported types have the same simple name, use a fully qualified name to disambiguate.

• Java does not support import aliases such as `import mypack.Message as Msg;`.

6. A multi-package example

This example separates student data, fee calculation and the application entry point.

```
src/
■■■ college/model/Student.java
■■■ college/service/FeeService.java
■■■ college/app/Main.java
```

Student.java

```
package college.model;

public class Student {
    private final String name;
    private final int credits;

    public Student(String name, int credits) {
        this.name = name;
        this.credits = credits;
    }
    public String getName() { return name; }
    public int getCredits() { return credits; }
}
```

FeeService.java

```
package college.service;
import college.model.Student;

public class FeeService {
    public int calculateFee(Student student) {
        return student.getCredits() * 1200;
    }
}
```

Main.java

```
package college.app;
import college.model.Student;
```

```
import college.service.FeeService;

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Aman", 4);
        FeeService service = new FeeService();
        System.out.println(s.getName() + " fee = " +
                           service.calculateFee(s));
    }
}
```

Compile and run

```
javac -d out src/college/model/Student.java src/college/service/FeeService.java src/college/app/Main.java
java -cp out college.app.Main
```

Expected output: `Aman fee = 4800`. The `model` package contains data, `service` contains business logic, and `app` starts the program.

7. Access modifiers and package visibility

Java's access modifiers control which code can use a class or member. Package visibility (also called default access) means no access modifier is written.

Modifier	Same class	Same package	Subclass in another package	Unrelated class in another package
private	Yes	No, except through accessible methods	No	No
No modifier (package-private)	Yes	Yes	No	No
protected	Yes	Yes	Yes, subject to subclass access rules	No
public	Yes	Yes	Yes, if the containing type is accessible	Yes, if the containing type is accessible

Example: the field `balance` below is package-private:

```
package mypack;
public class Account {
    double balance; // package-private
}
```

An unrelated class in another package cannot directly access `balance`. If other packages need access, expose an appropriate public method, such as a getter. Top-level classes can be public or package-private; they cannot be declared private or protected.

> **Protected nuance:** outside the package, `protected` access is available to subclasses under Java's subclass-access rules. It does not make the member public to every class in another package.

8. Subpackages: what they are and are not

`college`, `college.model` and `college.service` are distinct packages. A subpackage does not automatically inherit package-private access from its parent.

```
package college;          // one package
package college.model;    // a different package
```

A package-private member in `college` is not accessible merely because a class is in `college.model`. Also, `import college.*;` does not import `college.model.Student`; import the subpackage or class explicitly.

9. Compilation, classpath and execution

The **classpath** tells Java where to find compiled classes and JAR files. In the first example, `out` is the classpath root because it contains the package directories.

```
# macOS/Linux shell: compile all Java sources
find src -name "*.java" -print0 | xargs -0 javac -d out

# Run the entry point
java -cp out app.Main
```

Classpath separators: macOS/Linux generally use a colon (`out:lib/helper.jar`); Windows generally uses a semicolon (`out;lib\helper.jar`). The classpath should point to the directory above the package folder, not the package folder itself.

10. IDEs, Maven and Gradle

Using an IDE

• Create a Java project and a source folder.

• Use New → Package (menu wording varies) and enter a name such as `com.example.school`.

• Create classes inside the package; the IDE writes the declaration and puts files in the correct directory.

• Run the class containing `main`; the IDE handles compilation and classpath settings.

Maven convention

```
my-app/
■■■ pom.xml
■■■ src/
    ■■■ main/java/com/example/App.java
    ■■■ test/java/com/example/AppTest.java
```

The source root is `src/main/java`. A file at `com/example/App.java` should declare `package com.example;`.

```
mvn compile
mvn package
```

Gradle convention

```
src/main/java/com/example/App.java
src/test/java/com/example/AppTest.java
```

Maven and Gradle manage dependencies, compilation, tests and packaging. They do not change Java package rules.

11. Creating and using a JAR

A JAR (Java Archive) bundles compiled classes and resources so code can be distributed and reused.

Create a JAR

```
jar --create --file mypack.jar -C out mypack
```

This packages the `mypack` directory from the compiled output folder.

Compile and run an application using the JAR

```
javac -cp mypack.jar -d appout src/app/Main.java
java -cp mypack.jar:appout app.Main
```

Use a semicolon instead of a colon in the classpath on Windows. The JAR must contain the correct package path, such as `mypack/Message.class`.

> A library JAR is not automatically an executable JAR. Running with `java -jar` normally requires a manifest naming the main class and access to required dependencies.

12. Common errors and fixes

Error / symptom	Likely reason	Fix
package mypack does not exist	Compiler cannot find the package class.	Compile both source files together, or set classpath to compiled output/JAR.
Could not find or load main class	Wrong fully qualified name or classpath root.	Run `java -cp out app.Main`; check package and output folders.
Public class should be declared in a file named Message.java	Public class and filename differ.	Rename the file to `Message.java`.
Declared package does not match expected package	File path and package declaration disagree.	Move the file to the matching directory or correct the declaration.
Method is not public / cannot be accessed	Member is inaccessible from another package.	Make it public if it is intended to be part of the public API.
Duplicate class	Duplicate fully qualified class or incorrect source roots.	Remove/rename duplicate or fix project source paths.
Wildcard import does not resolve subpackage	Wildcard imports are not recursive.	Import the needed subpackage/type explicitly.

13. Best practices

• Use meaningful lowercase names such as `com.company.project.feature`.

• Group classes by responsibility rather than putting everything in one package.

• Expose only the public API that other packages need; keep implementation details private or package-private where appropriate.

• Avoid the default package in real projects.

• Keep source files under the configured source root and compiled files separate.

• Use Maven or Gradle when the project grows and needs dependencies or tests.

• Packages are namespaces and access-control boundaries, not a security sandbox.

14. Practice questions and answers

Questions

• 1. What is a user-defined package? Give two advantages.

• 2. Where should `package com.demo;` appear?

• 3. Difference between `import mypack.Message;` and `import mypack.*;`?

• 4. Does `import college.*;` import classes from `college.model`?

• 5. Why run `java -cp out app.Main` rather than `java Main`?

• 6. Can a subpackage access package-private members of its parent package?

• 7. What does `javac -d out` do?

• 8. What is a JAR and why is it useful?

Short answer key

• 1. A package groups related types; it helps with organization, naming conflicts, access control and reuse.

• 2. Before imports and type declarations, after any comments.

• 3. The first imports one type; the wildcard makes direct accessible types in that package available by simple name.

• 4. No. A subpackage is a separate package.

• 5. The class belongs to package `app`, so Java needs its fully qualified name and classpath root.

• 6. No, not by virtue of being a subpackage.

• 7. It writes compiled class files to the specified output directory and creates package folders as needed.

• 8. A JAR bundles classes/resources for distribution and reuse.

Mini-project

Create `tools.math.Calculator` with public methods `add(int a, int b)`, `subtract(int a, int b)` and `multiply(int a, int b)`. Then create `demo.Main` in a separate package, import the calculator, call all three methods, compile to an output folder, and run it.

15. Quick revision sheet

Concept	Remember
Package declaration	`package mypack;` — near the top of the source file
Directory mapping	`mypack.Message` → `mypack/Message.java` under source root
Specific import	`import mypack.Message;`
Wildcard import	Imports direct accessible types only; not subpackages
Compile	`javac -d out ...`
Run	`java -cp out app.Main`
Fully qualified name	Package name plus type name, e.g. `mypack.Message`
Package-private access	Available within the same package, subject to normal language rules
Subpackage	A distinct package; no automatic package-private access
JAR	Archive used to distribute classes and resources

Final takeaway: declare the package in each source file, keep the file under the matching directory, compile from the source root into an output folder, and run with the fully qualified class name and correct classpath.